

Exercise 1 — Atomicity: lỗi ở bước 2 thì bước 1 cũng bị hủy

Đoạn 1: Đưa transaction về trạng thái bình thường và xem dữ liệu ban đầu

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure for 'dvdrental'. The main pane shows a SQL query with line numbers 81 to 101. The query performs a join between 'film_category' and 'category', selects columns from 'ranked_films', and then rolls back the transaction. Below the query, the 'Data Output' tab shows a single row of results from the 'rental' table.

```
81 JOIN film_category AS fc
82 ON fc.film_id = f.film_id
83 JOIN category AS c
84 ON c.category_id = fc.category_id
85 )
86 SELECT
87 category_name,
88 film_id,
89 title,
90 length,
91 length_rank
92 FROM ranked_films
93 WHERE length_rank <= 3
94 ORDER BY category_name, length_rank, title;
95
96 --B4
97 ROLLBACK;
98
99 SELECT rental_id, return_date
100 FROM rental
101 WHERE rental_id = 2;
```

rental_id	return_date
2	2005-05-28 19:40:33

Total rows: 1 Query complete 00:00:00.135

Đoạn 2: Bắt đầu transaction và cập nhật tạm thời

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure for 'dvdrental'. The main pane shows a SQL query with line numbers 90 to 110. The query includes a SELECT statement for 'length' and 'length_rank' from 'ranked_films', followed by a WHERE clause, an ORDER BY clause, a comment '--B4', a ROLLBACK statement, and a transaction block starting with BEGIN and ending with COMMIT. The Data Output pane shows the results of the query, which is a single row with 'rental_id' 1 and 'return_date' '2026-09-18 15:14:26.853835'.

```
90      length,  
91      length_rank  
92  FROM ranked_films  
93  WHERE length_rank <= 3  
94  ORDER BY category_name, length_rank, title;  
95  
96  --B4  
97  ROLLBACK;  
98  
99  SELECT rental_id, return_date  
100 FROM rental  
101 WHERE rental_id = 2;  
102 BEGIN;  
103  
104  UPDATE rental  
105  SET return_date = CURRENT_TIMESTAMP  
106  WHERE rental_id = 2;  
107  
108  SELECT rental_id, return_date  
109  FROM rental  
110  WHERE rental_id = 2;
```

rental_id [PK] integer	return_date timestamp without time zone
1	2026-09-18 15:14:26.853835

Total rows: 1 Query complete 00:00:00.062 LF Ln 110, Col 21

Đoạn 3: Thực hiện bước thanh toán nhưng cố ý tạo lỗi

The screenshot shows the pgAdmin 4 interface with the following components:

- Object Explorer:** Displays the database structure for 'dvdrental' under 'PostgreSQL 18'. The 'public' schema is expanded, showing various objects like Aggregates, Collations, Domains, FTS Configurations, FTS Dictionaries, FTS Parsers, FTS Templates, Foreign Tables, Functions, Materialized Views, Operators, Procedures, Sequences, and Tables (15).
- Query Editor:** Contains a SQL query with line numbers 106 to 126. The query is as follows:

```
106 WHERE rental_id = 2;  
107  
108 SELECT rental_id, return_date  
109 FROM rental  
110 WHERE rental_id = 2;  
111  
112 INSERT INTO payment (  
113     customer_id,  
114     staff_id,  
115     rental_id,  
116     amount,  
117     payment_date  
118 )  
119 SELECT  
120     r.customer_id,  
121     1,  
122     r.rental_id,  
123     99.99,  
124     'WRONG_DATE_FORMAT'  
125 FROM rental AS r  
126 WHERE r.rental_id = 2;
```
- Query History:** Shows the executed query.
- Data Output / Messages / Notifications:** Displays an error message: "ERROR: current transaction is aborted, commands ignored until end of transaction block". Below the error, it shows "SQL state: 25P02".
- Status Bar:** At the bottom, it indicates "Total rows: 1", "Query complete 00:00:00.042", and the cursor position "Ln 126, Col 23".

Đoạn 4: Hủy toàn bộ transaction

The screenshot shows the pgAdmin 4 interface with the following components:

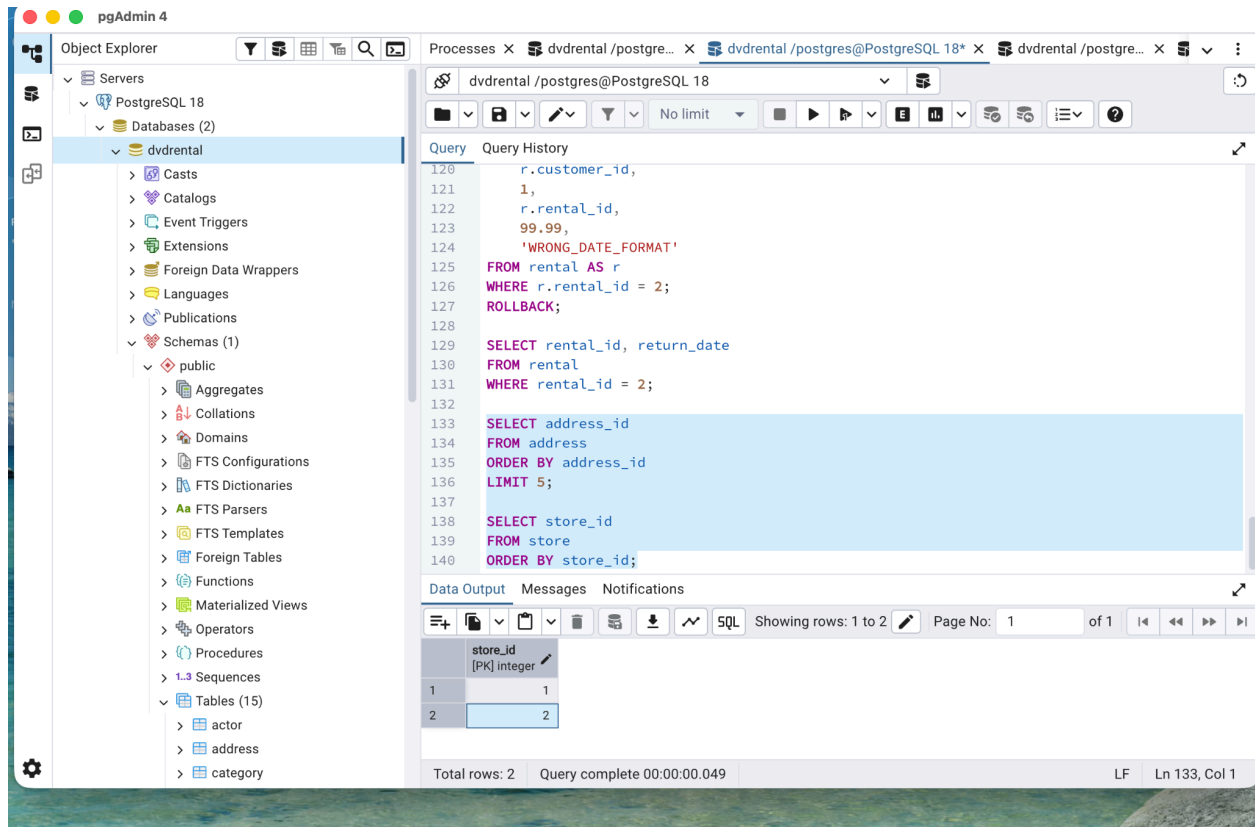
- Object Explorer:** Displays the database structure for 'PostgreSQL 18' and 'dvdrental'. The 'public' schema is expanded, showing various database objects like Aggregates, Collations, Domains, FTS Configurations, FTS Dictionaries, FTS Parsers, FTS Templates, Foreign Tables, Functions, Materialized Views, Operators, Procedures, Sequences, and Tables (15).
- Query Editor:** Contains the following SQL script:

```
111 INSERT INTO payment (  
112     customer_id,  
113     staff_id,  
114     rental_id,  
115     amount,  
116     payment_date  
117 )  
118  
119 SELECT  
120     r.customer_id,  
121     1,  
122     r.rental_id,  
123     99.99,  
124     'WRONG_DATE_FORMAT'  
125 FROM rental AS r  
126 WHERE r.rental_id = 2;  
127 ROLLBACK;  
128  
129 SELECT rental_id, return_date  
130 FROM rental  
131 WHERE rental_id = 2;
```
- Data Output:** Shows the results of the query. The first row is highlighted, showing the rental_id and return_date for rental_id = 2.

rental_id	return_date
2	2005-05-28 19:40:33
- Status Bar:** Indicates 'Total rows: 1' and 'Query complete 00:00:00.056'.

Exercise 2 — Savepoint: lỗi gán store nhưng staff vẫn được tạo

Đoạn 5: Kiểm tra dữ liệu hợp lệ để tạo staff



The screenshot shows the pgAdmin 4 interface with the following components:

- Object Explorer:** Displays the database structure. The 'dvdrental' database is selected, showing various objects like Casts, Catalogs, Event Triggers, Extensions, Foreign Data Wrappers, Languages, Publications, Schemas (1), and public schema objects (Aggregates, Collations, Domains, FTS Configurations, FTS Dictionaries, FTS Parsers, FTS Templates, Foreign Tables, Functions, Materialized Views, Operators, Procedures, Sequences, and Tables (15)).
- Query Editor:** Contains a SQL query with line numbers 120 to 140. The query is as follows:

```
120 r.customer_id,  
121 1,  
122 r.rental_id,  
123 99.99,  
124 'WRONG_DATE_FORMAT'  
125 FROM rental AS r  
126 WHERE r.rental_id = 2;  
127 ROLLBACK;  
128  
129 SELECT rental_id, return_date  
130 FROM rental  
131 WHERE rental_id = 2;  
132  
133 SELECT address_id  
134 FROM address  
135 ORDER BY address_id  
136 LIMIT 5;  
137  
138 SELECT store_id  
139 FROM store  
140 ORDER BY store_id;
```
- Data Output:** Displays the results of the query. The first two rows are highlighted in blue. The first row shows 'store_id' as 1, and the second row shows 'store_id' as 2. The column is labeled 'store_id [PK] integer'.
- Messages:** Shows the status of the query execution: 'Query complete 00:00:00.049'.

Đoạn 6: Tạo một staff mới trong transaction

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure for 'dvdrental', including Schemas (public) and Tables (actor, address, category). The main window shows a SQL query being executed in a transaction. The query is as follows:

```
142 BEGIN;  
143  
144 INSERT INTO staff (  
145     first_name,  
146     last_name,  
147     address_id,  
148     email,  
149     store_id,  
150     active,  
151     username,  
152     password  
153 )  
154 VALUES (  
155     'ACID',  
156     'Lab',  
157     1,  
158     'acid_lab_' || txid_current()::text || '@example.com',  
159     1,  
160     true,  
161     LEFT('acid_' || txid_current()::text, 16),  
162     'lab_password'
```

The Query History tab shows the query was executed successfully. The Messages tab displays the following output:

```
INSERT 0 1  
  
Query returned successfully in 55 msec.
```

The status bar at the bottom indicates the query was completed in 00:00:00.055, with the cursor at Line 163, Column 3.

Đoạn 7: Tạo savepoint và gây lỗi

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, with 'dvdrental' selected under 'PostgreSQL 18'. The main pane shows a SQL query with line numbers 154 to 175. The query includes a VALUES statement, a SAVEPOINT, an UPDATE, a ROLLBACK TO SAVEPOINT, another UPDATE, and a final WHERE clause. Below the query, the Messages tab shows an error: 'ERROR: smallint out of range' and 'SQL state: 22003'. The status bar at the bottom indicates 'Total rows: Query complete 00:00:00.038' and 'Ln 171, Col 1'.

```
154 VALUES (  
155 'ACID',  
156 'Lab',  
157 1,  
158 'acid_lab_' || txid_current()::text || '@example.com',  
159 1,  
160 true,  
161 LEFT('acid_' || txid_current()::text, 16),  
162 'lab_password'  
163 );  
164  
165 SAVEPOINT before_store_assignment;  
166  
167 UPDATE staff  
168 SET store_id = 99999  
169 WHERE username = LEFT('acid_' || txid_current()::text, 16);  
170  
171 ROLLBACK TO SAVEPOINT before_store_assignment;  
172  
173 UPDATE staff  
174 SET store_id = 99999  
175 WHERE username = LEFT('acid_' || txid_current()::text, 16);
```

ERROR: smallint out of range
SQL state: 22003

Total rows: Query complete 00:00:00.038 LF Ln 171, Col 1

Đoạn 9: Gán store hợp lệ và commit

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure: Servers > PostgreSQL 18 > Databases (2) > dvdrental. The main pane displays a SQL query with line numbers 165 to 186. The query is as follows:

```
165 SAVEPOINT before_store_assignment;
166
167 UPDATE staff
168 SET store_id = 99999
169 WHERE username = LEFT('acid_' || txid_current()::text, 16);
170
171 ROLLBACK TO SAVEPOINT before_store_assignment;
172
173 UPDATE staff
174 SET store_id = 99999
175 WHERE username = LEFT('acid_' || txid_current()::text, 16);
176
177 ROLLBACK TO SAVEPOINT before_store_assignment;
178 UPDATE staff
179 SET store_id = 1
180 WHERE username = LEFT('acid_' || txid_current()::text, 16);
181 COMMIT;
182 SELECT staff_id, first_name, last_name, store_id, username
183 FROM staff
184 WHERE username LIKE 'acid_%'
185 ORDER BY staff_id DESC
186 LIMIT 1;
```

Below the query editor, the Data Output tab shows the results of the query. The results are displayed in a table with the following columns: staff_id, first_name, last_name, store_id, and username. The table shows one row of data:

staff_id	first_name	last_name	store_id	username
1	4	ACID	Lab	1

The status bar at the bottom indicates "Total rows: 1" and "Query complete 00:00:00.062".

Question 1: Exercise 1 — Atomicity

Is the return_date updated? Why or why not?

No. The return_date was not updated permanently because the second INSERT statement failed. PostgreSQL marked the transaction as failed, and the ROLLBACK statement undid the previous UPDATE.

This demonstrates Atomicity: either all operations in a transaction succeed, or none of them are applied.

Question 2: Exercise 2 — Savepoints

What is the final result?

The staff member was successfully created and assigned to Store 1. The failed operation after the savepoint was rolled back, while the staff creation before the savepoint was preserved. Finally, the transaction was committed successfully.

In the local database, the staff.store_id foreign key constraint was not present. Therefore, the invalid store assignment did not produce the expected foreign-key error from the lecture slides.

Question 3: ACID Properties in the Banking Scenario

Which ACID property is responsible for each safety measure?

- a. The transfer must complete both the debit and the credit, or neither operation should take effect: Atomicity
- b. The account balance must never become negative: Consistency
- c. Transaction B must wait while Transaction A is updating the same account: Isolation
- d. The money remains available after a power failure immediately following a successful transaction: Durability

Question 4: Write-Ahead Logging

Why is writing to a log faster than writing directly to the main database files?

The WAL is written sequentially using an append-only process. Sequential writes are generally faster than writing to many random locations in the main database files.

Why should WAL be written sequentially?

Sequential writing reduces random I/O and latency. It also allows PostgreSQL to replay the log in order during crash recovery.

What is the trade-off of Durability?

Durability improves data safety, but it introduces additional disk I/O and latency because the WAL must be flushed to disk before a transaction is considered committed.

Question 5: Final Challenge — ANALYZE vs. VACUUM

What is the difference between ANALYZE and VACUUM?

ANALYZE collects statistics about data distribution and helps the query planner estimate row counts and choose an efficient execution plan. It does not remove dead tuples.

VACUUM cleans up dead row versions created by **UPDATE** and **DELETE**, allows storage space to be reused, and freezes old tuples to help prevent transaction ID wraparound.

They can be executed together:

VACUUM (ANALYZE) film;

How are they related to transaction history?

Because of MVCC, old row versions may remain visible to active transactions. Once no transaction needs those old versions, **VACUUM** can remove them. **ANALYZE** updates planner statistics so PostgreSQL understands the current state of the table.